

DESTROYER QUANTUM V27.1

Adaptive Compounding Engine

Task Brief & Engineering Specification

MQL4 | EURUSD H4 | Multi-Strategy System

1. YOUR ROLE AND MANDATE

You are a senior quantitative software engineer specializing in algorithmic trading systems. You have deep expertise in MQL4/MT4 development, institutional-grade risk management, portfolio theory, and adaptive position sizing. You have been brought in to perform a surgical upgrade to an existing production trading system.

This is not a casual task. The system you are working on has been backtested over 6 years of EURUSD data (January 2020 through April 2026), has generated over \$32,000 in net profit on a \$10,000 account, and is being prepared for live deployment. Every line of code you write must be production-grade. No shortcuts. No placeholders. No lazy implementations.

You will work through this task sequentially. Do not skip steps. Do not assume. Read the code. Understand the architecture. Then build. After each major step, verify your work before moving on.

2. SYSTEM CONTEXT - DESTROYER QUANTUM V27.1

DESTROYER QUANTUM is a multi-strategy algorithmic trading system running on MetaTrader 4 (MQL4). It operates on EURUSD on the H4 timeframe. The system deploys 8 independent strategy modules in parallel, each with its own magic number, entry logic, and risk profile.

2.1 Current Strategy Roster

The following strategies are active in the system. Each has a magic number used for trade identification:

- Phantom:** 205 trades | \$14,971 net | PF 1.60 | Primary alpha generator (46% of profits)
- NoiseBreakout:** 60 trades | \$11,278 net | PF 1.30 | Volatility squeeze breakout model
- Warden:** 22 trades | \$3,949 net | PF 1.82 | Best risk-adjusted performer
- Nexus:** 5 trades | \$1,200 net | PF 1.35 | Rare-event / extreme-condition strategy
- Silicon-X:** 137 trades | \$628 net | PF 1.40 | High-frequency microstructure scalper
- Mean Reversion:** 10 trades | \$84 net | PF 1.26 | Underutilized, needs parameter loosening
- Reaper Protocol:** 32 trades | \$62 net | PF 1.22 | Grid/martingale, negligible alpha
- Titan:** 5 trades | \$20 net | PF 2.53 | Effectively dormant, too restrictive

2.2 Overall Backtest Performance

Initial Deposit: \$10,000 | Final Equity: \$42,192.26 | Net Profit: \$32,192.26 (322%)
Profit Factor: 1.44 | Win Rate: 70.32% (334W / 141L) | Total Trades: 475
Max Drawdown: \$14,945.39 (39.57%) | Backtest Period: Jan 2020 - Apr 2026 (6.3 years)
Largest Win: \$4,429.92 | Largest Loss: -\$3,156.41 | Avg Win: \$313.85 | Avg Loss: -\$515.14

2.3 V26 vs V27.1 Comparison

V26 Results (same 6-year backtest, EURUSD H4):

Net Profit: \$5,463.58 | Profit Factor: 2.35 | Max DD: Not available from log
473 trades | Final Equity: \$15,463.58

V27.1 Results:

Net Profit: \$32,192.26 | Profit Factor: 1.44 | Max DD: 39.57%
475 trades | Final Equity: \$42,192.26

The V27.1 generates nearly 6x more profit than V26 with a similar trade count. The profit factor decreased from 2.35 to 1.44, which indicates that V27.1 is taking on more risk per unit of return. The 39.57% max drawdown is the primary concern that the new lot sizing system must address.

3. THE PROBLEM - CURRENT LOT SIZING IS ORDINARY

The current lot sizing system in DESTROYER QUANTUM V27.1 is functional but not adaptive. It uses static, hardcoded risk multipliers that do not respond to real-time strategy performance. This is the single biggest weakness in the system architecture.

3.1 Current Implementation (MoneyManagement_Quantum)

The primary lot sizing function is MoneyManagement_Quantum(). It works as follows:

- Reaper (magic 888001/888002): hardcoded 3.0x risk multiplier
- Silicon-X (magic 984651): hardcoded 1.0x risk multiplier
- Warden: hardcoded 0.5x risk multiplier
- Titan / Mean Reversion / Others: hardcoded 0.2x risk multiplier

The lot size is then calculated as:

$\text{riskAmount} = \text{AccountEquity} \times (\text{baseRiskPercent} \times \text{riskMultiplier}) / 100$
 $\text{rawLots} = \text{riskAmount} / (50 \text{ pips} \times \text{tickValue})$

These multipliers are set once and never change. They do not account for:

- Recent win/loss streaks per strategy
- Rolling profit factor over a lookback window
- Current market regime (trending vs ranging vs volatile)
- Time since last win or last loss
- Correlation between concurrent strategy positions
- Whether a strategy is currently "hot" or "cold"

3.2 Additional Lot Sizing Functions

The system also contains several other lot sizing functions that were introduced in earlier versions but are either partially used or superseded:

- GetLotSize_Ascension(): Uses Kelly Criterion with hardcoded win rate (0.7922) and odds ratio (3.81). Has compounding modes (Aggressive Growth / Capital Preservation) based on drawdown and streaks. Uses fractional Kelly

(25%). This is the closest thing to an adaptive system but it uses global statistics, not per-strategy statistics.

- GetKellyLotSize(): Another Kelly-based function with hardcoded conservative estimates (65% win rate, 50/40 avg win/loss). Uses quarter-Kelly with 5% equity cap. Generic, not strategy-aware.
- GetLotSizeV8_5_9_FIXED(): The original V8 lot sizing. Risk-based with portfolio budget check. No performance adaptation.
- V23_CalculateLotSize(): Uses empirical probability, tail-risk dampening, and R-expectancy. This is the most sophisticated function but it operates on strategy indices, not magic numbers, and its adaptation is limited to halving lots on negative R-expectancy.
- GetGeneticRiskMultiplier(): Tiered risk based on profit factor thresholds. $PF < 1.0 = 0\%$ (kill), $PF 1.0-1.3 = 20\%$, $PF 1.3-2.0 = 100\%$, $PF \geq 2.0 = 200\%$. Binary and reactive, not smooth.
- GetStrategySpecificRisk(): Hardcoded magic-number-to-risk mapping. Overrides genetic check for known strategies. Same problem as MoneyManagement_Quantum - static multipliers.

None of these functions communicate with each other. There is no unified adaptive layer.

3.3 Why This Matters - The Hedge Problem

Consider this scenario which happens regularly in the current system:

- Phantom opens a BUY with 1.0 lot size and wins +\$300
- NoiseBreakout opens a SELL with 1.0 lot size and loses -\$500
- Net result: -\$200

Both strategies used the same lot size because the system does not know which one is currently performing better. If the system had an adaptive compounding engine that recognized Phantom was on a win streak and NoiseBreakout was on a loss streak, it could have allocated:

- Phantom BUY: 1.5 lots -> wins +\$450
- NoiseBreakout SELL: 0.3 lots -> loses -\$150
- Net result: +\$300

Same market conditions. Same strategies. Different lot sizes. That is the power of adaptive compounding.

4. YOUR ASSIGNMENT - BUILD THE ADAPTIVE COMPOUNDING ENGINE

You are to design and implement a complete Adaptive Compounding Engine for DESTROYER QUANTUM V27.1. This engine will replace the current static lot sizing logic with a dynamic, performance-aware system that allocates capital intelligently across all 8 strategies.

4.1 Design Principles

The engine must follow these principles without exception:

- Winning strategies get more capital. Not recklessly - the increase must be bounded and gradual.
- Losing strategies get less capital. Not zero immediately - there is a probation period before kill.
- Every strategy starts with equal allocation. Performance history is built from the first trade.
- The system must be self-correcting. If a previously winning strategy starts losing, it must be de-allocated.
- The system must respect the existing portfolio risk budget (InpMaxTotalRisk_Percent = 5.0%).
- All lot sizes must be normalized to broker lot step, min lot, and max lot requirements.
- The engine must be pure MQL4. No external libraries. No DLL imports. Compile-ready.

4.2 Core Architecture - Per-Strategy Performance Tracker

You must first build a per-strategy performance tracking system. This is the foundation that the entire adaptive engine depends on. Create a struct or series of arrays that track the following for each strategy (identified by magic number):

1. Rolling Win Rate (exponentially weighted, last 50 trades)
2. Rolling Profit Factor (exponentially weighted, last 50 trades)
3. Rolling R-Expectancy (average R-value of last 50 trades)
4. Current Win/Loss Streak (consecutive wins or losses)
5. Time Since Last Trade (bars since last close for this strategy)
6. Cumulative Net Profit (total P&L for this strategy since inception)
7. Last 10 Trade Average Profit (short-term performance pulse)
8. Max Drawdown Experienced by this strategy (peak-to-trough)
9. Current Equity Curve Slope (is this strategy trending up or down in equity)
10. Confidence Score (composite score derived from the above metrics)

Use exponential weighted moving average (EWMA) with $\alpha = 0.05$ for the rolling calculations. This gives more weight to recent trades while still maintaining historical context.

4.3 Core Architecture - The Adaptive Lot Sizing Function

Create a single unified function that replaces all current lot sizing. The function signature should be:

```
double AdaptiveCompound_GetLotSize(int magicNumber, double stopLossPips)
```

This function must execute the following steps in order:

- STEP 1: Retrieve the performance tracker for this magic number.
- STEP 2: Calculate the Confidence Score (see section 4.4).
- STEP 3: Determine the Allocation Tier (see section 4.5).
- STEP 4: Calculate the base lot size using fractional Kelly Criterion with strategy-specific win rate and R-ratio.
- STEP 5: Apply the Allocation Tier multiplier.
- STEP 6: Apply the Regime Multiplier (current market regime adjustment).
- STEP 7: Apply the Correlation Discount (if multiple strategies are open in opposite directions).
- STEP 8: Apply the Drawdown Brake (reduce size if portfolio DD exceeds thresholds).
- STEP 9: Enforce per-strategy min/max lot limits.
- STEP 10: Normalize to broker lot step and validate against portfolio risk budget.
- STEP 11: Log the decision with full reasoning for audit trail.
- STEP 12: Return the final lot size.

4.4 Confidence Score Calculation

The Confidence Score is a value between 0.0 and 1.0 that represents how much the system "trusts" a strategy right now. It is calculated as a weighted composite:

$$\text{ConfidenceScore} = (w1 \times \text{RollingWinRate}) + (w2 \times \text{NormalizedPF}) + (w3 \times \text{StreakFactor}) + (w4 \times \text{EquitySlope}) + (w5 \times \text{RecencyFactor})$$

Where:

$w1 = 0.30$ (win rate is important but not everything)

$w2 = 0.25$ (profit factor shows edge quality)

$w3 = 0.20$ (streaks indicate current market alignment)

$w4 = 0.15$ (equity curve slope shows trajectory)

$w5 = 0.10$ (recency ensures dormant strategies decay)

$\text{NormalizedPF} = \text{clamp}((\text{RollingPF} - 1.0) / 2.0, 0.0, 1.0)$ -> PF of 1.0 = 0, PF of 3.0 = 1.0

$\text{StreakFactor} = \text{clamp}(\text{StreakValue} / 10.0, -1.0, 1.0)$ -> positive for wins, negative for losses

$\text{EquitySlope} = \text{clamp}((\text{Last10AvgProfit} / \text{AbsAvgTrade}), -1.0, 1.0)$

$\text{RecencyFactor} = \text{clamp}(1.0 - (\text{BarsSinceLastTrade} / 200.0), 0.0, 1.0)$ -> decays over 200 bars

The Confidence Score determines everything. A strategy with 0.8 confidence gets significantly more capital than one with 0.3 confidence. The transition must be smooth, not binary.

4.5 Allocation Tiers

Based on the Confidence Score, each strategy falls into one of 5 allocation tiers:

ELITE (Confidence ≥ 0.75): 2.0x - 3.0x base allocation

These strategies are firing on all cylinders. Maximum capital deployment.

STRONG ($0.55 \leq \text{Confidence} < 0.75$): 1.2x - 1.8x base allocation

Performing well. Above-average allocation.

NEUTRAL ($0.35 \leq \text{Confidence} < 0.55$): 0.8x - 1.2x base allocation

Normal performance. Standard allocation.

WEAK ($0.15 \leq \text{Confidence} < 0.35$): 0.3x - 0.7x base allocation

Underperforming. Reduced allocation. On probation.

KILL (Confidence < 0.15): 0.0x allocation (trade blocked)

Consistently losing. Strategy is shut down until manual review or confidence recovery.

The multiplier within each tier should be interpolated linearly. For example, a confidence of 0.65 in the STRONG tier should yield approximately 1.5x allocation (midpoint of 1.2x-1.8x).

4.6 Regime Multiplier

The system already has regime detection (V23_DetectMarketRegime). Use the current regime to adjust lot sizes:

TRENDING regime: 1.2x multiplier for trend strategies (Phantom, Warden, Titan)
0.7x multiplier for mean-reversion strategies (Mean Reversion)

RANGING regime: 0.8x multiplier for trend strategies
1.3x multiplier for mean-reversion strategies

VOLATILE regime: 0.6x multiplier for all strategies (capital preservation)

UNKNOWN regime: 1.0x multiplier (neutral)

The regime multiplier must never exceed 1.5x or go below 0.3x regardless of regime.

4.7 Correlation Discount

When multiple strategies have open positions in opposite directions simultaneously, the system is effectively hedged. This is not inherently bad, but it means the net exposure is lower than the gross exposure. The adaptive engine must account for this:

If net open positions are in the same direction: No discount (1.0x)

If net open positions are hedged (opposite directions):

$\text{HedgeRatio} = \text{Min}(\text{GrossLong}, \text{GrossShort}) / \text{Max}(\text{GrossLong}, \text{GrossShort})$

$\text{CorrelationDiscount} = 1.0 - (\text{HedgeRatio} \times 0.3)$

This means if you have 1.0 lot long and 0.8 lots short, the HedgeRatio is 0.8, and the discount is 0.76x. This prevents excessive capital being tied up in hedged positions that cancel each other out.

4.8 Drawdown Brake

When the portfolio is in drawdown, all strategies must reduce risk proportionally:

Portfolio DD < 5%: No brake (1.0x)

Portfolio DD 5-10%: 0.8x multiplier

Portfolio DD 10-20%: 0.5x multiplier

Portfolio DD 20-30%: 0.3x multiplier

Portfolio DD > 30%: 0.1x multiplier (emergency preservation)

This is the single most important safety mechanism. The current system hit 39.57% max drawdown. This brake alone should reduce that significantly.

4.9 Per-Strategy Min/Max Limits

Each strategy must have its own lot size boundaries to prevent runaway allocation:

Minimum lot size: 0.01 lots (broker minimum)

Maximum lot size per strategy: Must not exceed 30% of portfolio risk budget

Maximum lot size absolute cap: 5.0 lots (prevents any single strategy from dominating)

These limits must be configurable via extern input parameters so they can be tuned in backtesting.

5. IMPLEMENTATION STEPS (EXECUTE IN ORDER)

Follow these steps exactly. Do not skip any. Verify after each step.

Step 1: Read and Understand the Full Codebase

Before writing a single line of code, you must read and understand the following sections of DESTROYER_V27_1_PATCHED.mq4:

- All extern/input parameters (lines 1000-1070 approximately)
- GetLotSize_Ascension() function (around line 3368)
- GetKellyLotSize() function (around line 4711)
- MoneyManagement_Quantum() function (around line 11673)
- V23_CalculateLotSize() function (around line 12613)
- GetGeneticRiskMultiplier() function (search for it)
- GetStrategySpecificRisk() function (around line 4237)
- ValidateTradeRisk() function (around line 4510)
- DetermineCompoundingMode() function (around line 3455)
- The V23_StrategyPerformance struct and all V23 functions

Understand how these functions interact. Identify every location where lot sizes are calculated or modified. Document the call chain for each strategy.

Step 2: Build the Performance Tracker

Create the per-strategy performance tracking system. This includes:

- A struct (ACS_StrategyPerf) to hold all per-strategy metrics
- An initialization function that sets up trackers for all 8 strategies using their magic numbers
- An update function called after every trade close that recalculates all rolling metrics
- A function to calculate the Confidence Score from the metrics
- A function to determine the Allocation Tier from the Confidence Score

The tracker must persist across ticks (use global/static variables). It must handle the case where a strategy has zero trades gracefully (return neutral confidence of 0.5).

Step 3: Build the Core Adaptive Function

Implement AdaptiveCompound_GetLotSize() following the 12-step process described in section 4.3. This function must:

- Accept magic number and stop loss in pips as parameters
- Return a normalized lot size (or 0 to block the trade)
- Log every decision with full reasoning (confidence score, tier, multiplier chain, final lot)
- Handle edge cases (zero stop loss, zero equity, invalid magic number)
- Be deterministic (same inputs must always produce same output)

Build it modular. Each step (confidence, tier, kelly, regime, correlation, drawdown brake) should be its own helper function that can be tested independently.

Step 4: Wire It Into the Existing System

Replace every call to `MoneyManagement_Quantum()`, `GetKellyLotSize()`, `GetLotSize_Ascension()`, `V23_CalculateLotSize()`, and `GetLotSizeV8_5_9_FIXED()` with `AdaptiveCompound_GetLotSize()`.

This means you need to find every strategy execution block (Phantom, NoiseBreakout, Warden, Nexus, Silicon-X, Mean Reversion, Reaper, Titan) and update the lot calculation line.

IMPORTANT: Do NOT delete the old functions. Comment them out with a note: `"// REPLACED BY ADAPTIVE COMPOUNDING ENGINE - [date]"`. This preserves rollback capability.

Also update the `OnTradeClose` handler to call the performance tracker update function after every closed trade.

Step 5: Add Configuration Parameters

Add the following extern input parameters at the top of the file (in the input section):

```
extern bool  InpACS_Enabled = true;           // Enable Adaptive Compounding
extern double InpACS_EWMA_Alpha = 0.05;       // EWMA alpha for rolling stats
extern int   InpACS_LookbackTrades = 50;      // Rolling window trade count
extern double InpACS_KellyFraction = 0.25;    // Fractional Kelly (25% = quarter Kelly)
extern double InpACS_MinLotSize = 0.01;       // Minimum lot size per strategy
extern double InpACS_MaxLotSize = 5.0;        // Maximum lot size per strategy
extern double InpACS_MaxStrategyRiskPct = 30.0; // Max % of portfolio risk per strategy
extern double InpACS_KillThreshold = 0.15;    // Confidence below this = strategy killed
extern double InpACS_EliteThreshold = 0.75;   // Confidence above this = elite tier
extern double InpACS_MinRegimeMultiplier = 0.3; // Minimum regime multiplier
extern double InpACS_MaxRegimeMultiplier = 1.5; // Maximum regime multiplier
extern bool   InpACS_EnableCorrelationDiscount = true; // Enable hedge discount
extern bool   InpACS_EnableDrawdownBrake = true; // Enable DD brake
extern bool   InpACS_VerboseLogging = true;    // Detailed audit log
```

If `InpACS_Enabled` is false, the system must fall back to the original `MoneyManagement_Quantum()` logic. This is critical for safe deployment.

Step 6: Verify Compilation

After all changes, the file must compile without errors or warnings in MetaEditor. Common issues:

- Duplicate function names (the old functions are commented out, not deleted)
- Missing struct member access (ensure the perf struct is properly initialized)
- Array out of bounds (strategy index must be validated before array access)
- Implicit type conversions (use explicit casts where needed)
- Uninitialized variables (all doubles must be initialized to 0.0)

Fix every warning. Warnings in MQL4 often become runtime bugs.

Step 7: Write a Summary of Changes

At the end of your work, provide a complete summary document that includes:

1. List of every function added (name, purpose, line number)
2. List of every function modified (what changed and why)
3. List of every function replaced (old vs new, with reasoning)
4. The complete parameter list with recommended values for initial backtesting
5. Any known limitations or areas for future improvement
6. Expected impact on the backtest (lot distribution, risk reduction, profit shift)

This summary will be used for code review before backtesting.

6. WHAT NOT TO DO

- Do NOT use hardcoded strategy names or magic numbers in the adaptive logic. The system must work with any magic number dynamically.
- Do NOT implement martingale or anti-martingale. The engine is adaptive compounding, not bet doubling.
- Do NOT create a system where a single strategy can risk more than 30% of the total portfolio budget.
- Do NOT remove or disable the existing `ValidateTradeRisk()` or `ApproveTrade()` functions. They are safety gates.
- Do NOT add any external dependencies, DLL imports, or non-standard MQL4 libraries.
- Do NOT use global variables that conflict with the existing V23, V24, V25, V26, or V27 variable namespaces.
- Do NOT implement this as a "set and forget" system. It must update its beliefs after every trade close.
- Do NOT make the Confidence Score jump wildly. The EWMA smoothing is intentional - it prevents overreaction to single trades.
- Do NOT ignore the regime multiplier. A strategy that works in trending markets should not get the same allocation in ranging markets.
- Do NOT skip the logging. Every lot size decision must be traceable in the MT4 journal.

7. SUCCESS CRITERIA

The Adaptive Compounding Engine is successful if, after backtesting:

- The max drawdown decreases by at least 10% (from 39.57% to below 35%, target below 30%).
- The net profit does not decrease by more than 20% (still above \$25,000 on \$10K).
- Winning strategies (Phantom, Warden, NoiseBreakout) receive visibly larger lot sizes than losing strategies.
- Losing strategies (Reaper, Titan) receive visibly smaller lot sizes or are blocked entirely.
- The system self-corrects during drawdown periods (lot sizes shrink during the Oct 2021 - Apr 2022 DD).
- The profit factor improves or remains stable (target ≥ 1.40).
- The code compiles cleanly, runs without errors, and produces detailed logs.

8. FINAL WORD

This is the most important upgrade DESTROYER QUANTUM has received. The difference between a good trading system and a great one is not the entry signals - it is the position sizing. The current system has 8 strategies generating

alpha. What it lacks is the intelligence to allocate capital where it is working and starve capital where it is not.

You are building the brain that sits on top of the brawn. Make it count.

Work sequentially. Verify after each step. Do not rush. Do not skip. The backtest will expose every shortcut you take.

Begin.